

[...]

Live coding fuses the role of the musical improviser with that of the computer programmer. Compared with much performance of music on laptops, live coding is typically both abstract and revelatory. Its abstractness (especially from the audience's perspective) consists of the performers' reaching down deeper into the technology, creating sound in real time by typing text in a programming language rather than by manipulating the graphical user interface of a commercial software application. On the other hand, its revelatory nature is embedded in the dictum "Show us your screens": The pro-programmers' laptop screens are often projected for the audience to observe every keystroke. Not all audience members may understand the code, but they will understand that the

performers are not (as the well-worn joke goes) checking e-mail. As for technically knowledgeable observers, they may appreciate a coder's technological dexterity in the highly exposed context of live performance, just as traditional concertgoers appreciate a virtuoso.

[...]

—Douglas Keislar

About This Issue, p. 1

Magnusson, Thor: „Herding Cats: Observing Live Coding in the Wild“, Computer Music Journal 38/1 (2014), S. 8–16, Auszüge.

Thor Magnusson

Department of Music
University of Sussex
Falmer, Brighton, BN1 9QJ, UK
T.Magnusson@sussex.ac.uk

Herding Cats: Observing Live Coding in the Wild

Abstract: After an eventful decade of live-coding activities, this article seeks to explore the practice with the aim of situating it in the history of contemporary arts and music. The article introduces several key points of investigation in live-coding research and discusses some examples of how live-coding practitioners engage with these points in their system design and performances. In light of the extremely diverse manifestations of live-coding activities, the problem of defining the practice is discussed, and the question is raised whether live coding is actually necessary as an independent category.

[...]

Live coding is a heterogeneous practice and thus somewhat hard to define. It involves a multiplicity of approaches that have one thing in common: Algorithmic instructions are written in real time. Collins (2011, p. 209) states that "the more profound the live coding, the more a performer must confront the running algorithm, and the more significant the intervention in the works, the deeper the coding act." For Collins, most performances fail to "live up to this promise." The argument, also expressed by McLean (2008), is that a true "liveness" requires that the performer is not simply manipulating pre-written code in real time, but is actually writing and

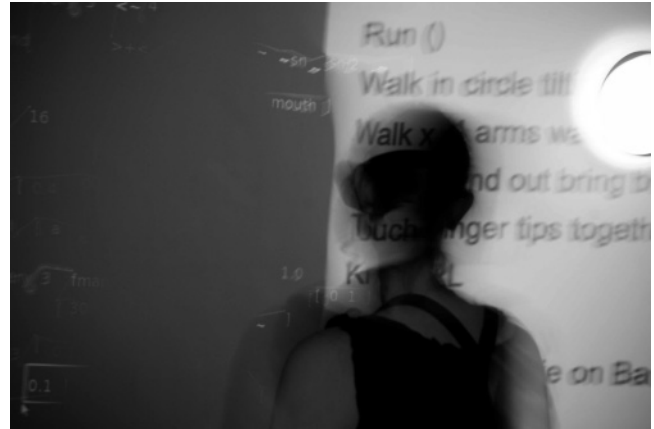
Figure 1. Kate Sicchio dancing with code.
(Photograph by Bill Best, Sheffield, UK, Creative Commons BY-NC-ND 3.0.)

modifying algorithms during the execution of the program. This is arguably a prerequisite of live coding, as simply running code and changing parameters in prewritten code is more comparable to operating buttons, sliders, and knobs on a screen interface or a MIDI controller. We could talk about weak and strong definitions of live coding, with the weak one allowing trivial code manipulations and strong one adhering to Collins's and McLean's position. The strong definition would result in the conclusion that some so-called live-coding performances do not really include any live coding at all.

In the absence of any clear definitions of live coding, apart from the requirement that algorithms are written in real time, it may be appropriate to take a sample of some prolific live coders and their systems, and ask whether we can detect an aesthetic style, or any specific performance contexts or technological foundations. The examples here are chosen with the aim of demonstrating the variety of the practice, leaving out other significant contributors to live coding whose performance setup might be similar. The aim is to study how live coding operates in a live context, and to outline the problems practitioners are trying to address in their work.

PowerBooks.UnPlugged is an ensemble of six performers who make use of wireless networks to communicate and share code in a networked live-coding performance. As the name indicates, the laptops are unplugged, and the only sound output is from the built-in laptop speakers. Members of the collective sit among the audience, blurring the traditional performer–audience spatial divide, while sending code snippets to each other for collaborative coding (Rohrhuber et al. 2007). The ensemble describes itself as “the first acoustic computer music folk band” (for instance, on the band Web site at pbup.goto10.org) with a reference to the demystification of the laptop as a musical instrument.

One of the early live-coding collectives is a group called Slub, consisting of three members who perform with different live-coding systems of their own creation and who collaborate over a network using a shared musical time clock. Their systems are well known for their experimental and innovative aspects, focusing on language design and human–computer interaction. **Slub** regularly



uses multiple projectors to overlay the performers' desktop images, an act of obfuscation where the rendered image on the wall becomes a visual montage of an aesthetic dimension. The collective often collaborates with other artists, recently with performance artist **Hester Reeves**.

[...]

Figure 3. Benoît and the Mandelbrots playing their greatest hits. (Photograph by Daniel Bollinger, www.danielbollinger.de.)



[...]

There is also a boy-band dimension in live coding, most notably present in the **collective Benoît** and the **Mandelbrots** (see Figure 3). This multilingual (both in terms of human and machine languages) supergroup continually surprises with their advanced systems, coding expertise, and solutions to the problems of networked computer music and collaborative audiovisual performance.

Andrew Sorensen and Andrew Brown perform under the name **aa-cell** (Sorensen and Brown 2007). They use a Scheme-based environment called **Impromptu**, written by Sorensen (2005). Their performances have impressed audiences around the world through their range of expression, coding eloquence, and compositional structures. If one were to name virtuoso live coders, algorithmic equivalents to Paganini and Liszt, the names of these performers are bound to be included. Sorensen is currently working on a new live-coding environment called **Extempore for “cyber-physical programming”** (Sorensen and Gardner 2010).

[...]

The list of live-coding practitioners enumerated here could be much longer, but it sufficiently demonstrates that there are no specific tools, practices, or musical aesthetics at play: Live coding is a fuzzy concept representing a performance technique with a multitude of practices that do not share any one essential requirement, except perhaps that algorithms are written and operated on in real time. Each of the named practices addresses specific problems involved in composing with algorithms, some of which could be summarized as follows:

- Languages for algorithmic thinking
- Comprehensibility and intuitiveness of code
- Audience engagement and involvement
- Graphical visualizations of algorithms
- Virtuosity and the system’s provision for speed coding
- Compositional expressiveness
- Direct access to the artistic material (e.g., synthesis or temporal patterns)
- Collaboration through network protocols
- “Liveness” and the ability to control existing structures
- Embodiment and physicality
- Live-coding systems as musical pieces or works of art

As a novel research field with a strong practical underpinning, live coding engages with diverse unexplored and novel areas in (musical) composition, human–computer interaction, programming-language design, and performance studies, rendering the practice an ideal platform for both artistic and scientific experiments. The examples in this article illustrate **how live coding tries to communicate algorithmic thinking, real-time composition, and networked collaborations to the audience by designing innovative systems or exploring novel performance contexts**. These examples typify approaches to humanizing machines by creating conversational interfaces with them, enabling artists to issue commands through the notation of code. [...]